

CSI CYBER

ANÁLISE ESTÁTICA DE MALWARES



4

LISTA DE FIGURAS

Figura 1 – Esboço de um programa na RAM	11
---	----



LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Código em C com chamada de função	13
--	----

EMSE

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Calculando hash de arquivos.....	7
Comando de prompt 2 – Gerando imphash	9

EMANIP

SUMÁRIO

1 ANÁLISE ESTÁTICA DE MALWARES	6
2 ANÁLISE ESTÁTICA BÁSICA.....	6
2.1 Função Hash	6
2.2 Imphash.....	8
2.3 VírusTotal	9
2.4 Ferramenta PEiD.....	10
2.5 Ferramenta pestudio	10
3 ANÁLISE ESTÁTICA AVANÇADA	11
3.1 Função	11
CONCLUSÃO.....	13
REFERÊNCIAS	15

1 ANÁLISE ESTÁTICA DE MALWARES

O objetivo deste capítulo é apresentar algumas formas de análises estáticas de malware, básicas e avançadas.

Os *samples* estarão disponíveis para o aluno baixar e usar apenas em sua máquina virtual, a qual deverá estar isolada da sua máquina *host* e com o snapshot executado antes dos testes. Os arquivos terão nomes genéricos, pois malwares verdadeiros não se identificam com nomes que indicam o que eles desejam fazer.

2 ANÁLISE ESTÁTICA BÁSICA

Assim que rebemos um artefato para análise, a primeira coisa que devemos fazer é solicitar informações de como e quando esse artefato foi encontrado, bem como o sistema operacional, perfil do usuário, entre outras que o analista julgar indispensáveis para uma análise inicial.

Porém, para nossa disciplina, os artefatos disponibilizados não terão informação alguma, geraremos nossa informação acerca de cada artefato a partir dos dados coletados durante as análises.

Uma das premissas da análise estática é não executar o malware, toda informação gerada será obtida de modo que se analise apenas o binário do malware, em busca de informações básicas, como detecção por antivírus, *hashes*, strings, estrutura do binário, chamadas de funções da API do sistema operacional.

2.1 Função Hash

Conhecida apenas por *Hash*, é uma função que faz o mapeamento de dados com comprimento variável em dados de comprimento fixo. É impossível retornar aos dados iniciais utilizando esses valores de comprimento fixo. Podemos utilizar o *hash* para diversos cenários, não só na análise de malware, por exemplo: após baixar uma imagem .iso de um sistema operacional, geralmente 3,0 GB de dados, podemos

conferir se todo o conteúdo da imagem .iso foi baixado corretamente por meio de uma função *hash*, pois o site da imagem gerou o seu *hash* e o deixou disponibilizado em seu site para que os usuários possam conferir se o que foi gerado localmente é exatamente o que se encontra no site.

Para a análise de malware, o *hash* identificará o malware a partir dessa informação, como uma espécie de impressão digital. Existem diversos tipos de funções *hashes*, entre os quais, a MD5 é a mais utilizada, por ser o algoritmo mais rápido, porém já aconteceram casos em que foi possível gerar o mesmo *hash* MD5 para dois arquivos diferentes.

Vejamos no comando abaixo o cálculo dos seguintes hash: MD5, SHA1 e SHA256. Com a ajuda do comando *time*, podemos observar o tempo que leva para cada algoritmo processar uma imagem .iso de 4,7G.

```
hugo@ubuntu:~/Downloads$ ls -lh Win10_2004_BrazilianPortuguese_x64.iso
-rw-rw-r-- 1 hugo hugo 4,7G ago 21 04:34 Win10_2004_BrazilianPortuguese_x64.iso

hugo@ubuntu:~/Downloads$ time md5sum Win10_2004_BrazilianPortuguese_x64.iso
9cc397e2cbd9270fde0aee12347d00fc Win10_2004_BrazilianPortuguese_x64.iso

real    0m10,442s
user    0m2,838s
sys     0m7,526s

hugo@ubuntu:~/Downloads$ time sha1sum Win10_2004_BrazilianPortuguese_x64.iso
88f85988436182b73249136e6ce72883657237ec Win10_2004_BrazilianPortuguese_x64.iso

real    0m11,885s
user    0m3,051s
sys     0m8,805s

hugo@ubuntu:~/Downloads$ time sha256sum Win10_2004_BrazilianPortuguese_x64.iso
9f8ab79dcc6628c6df6ec633890ea841ab2446ba25a414e7bd947d75bac82ccd
Win10_2004_BrazilianPortuguese_x64.iso

real    0m25,517s
user    0m9,766s
sys     0m15,691s
```

Comando de prompt 1 – Calculando hash de arquivos

Fonte: Elaborado pelo autor (2020)

O comando *md5sum* levou menos da metade do tempo em relação ao *sha256sum*.

Os *hashes* são completamente alterados apenas com o acréscimo de um ponto no final do texto.

Por essa razão, utilizamos *hashes* para identificar os malwares, a maioria das vezes precisamos buscar por malwares em sites ou buscar algumas análises já feitas por outros analistas, então, a opção de pesquisa será por *hash* md5, sha256 ou sha1, pois você terá certeza de que o malware que está analisando é exatamente igual ao que foi encontrado por outro analista.

2.2 Imphash

Imphash (do inglês *Import Hashing*) é uma técnica desenvolvida para identificar malwares pelas importações das DLLs do sistema operacional. O *hash* é calculado com base nos nomes da biblioteca e na ordem em que as funções são importadas pelo malware utilizando a API do S.O.

Por exemplo, quando um programa quer escrever um arquivo no disco, ele não precisa implementar uma nova função que faça isso, mas, sim, usar a função disponibilizada pelo sistema operacional para executar tal tarefa. Não importa o que o malware faz, mas, se existem dois arquivos que usam as mesmas funções e forem compilados na mesma fonte, eles tendem a ter o mesmo imphash.

“Ah, Hugo, mas você falou que se eu alterar apenas um byte no arquivo o hash também será alterado!” Isso mesmo, porém, no imphash, estamos falando de uma *string* que é gerada a partir da tabela de importação de bibliotecas e a ordem em que suas funções são chamadas. Vamos a mais um exemplo:

Um binário que mostra, na saída padrão, um simples “Hello Word”, no código terá apenas uma importação de biblioteca “`#include <stdio.h>`” que usará a Kernel32.dll, e dentro dessa biblioteca teremos algumas funções que serão necessárias para a execução do código. Vamos tomar por exemplo apenas duas funções, DeleteCriticalSection() e EnterCriticalSection(), exatamente nessa ordem.

A *string* terá o padrão abaixo:

biblioteca1.funcao1,biblioteca1.funcao2,biblioteca2.funcao1

A *string* deverá seguir a ordem:

1) Remover a extensão da biblioteca (KERNEL32.dll vira KERNEL32), acrescentar o nome da função ao nome da biblioteca separada por ponto;

- 2) Cada import (biblioteca.funcao) deverá ser separado por vírgula;
- 3) Transformar tudo em minúsculo e calcular o hash.

Então, no nosso exemplo, teremos a seguinte *string*:

kernel32.deletcriticalsection,kernel32.entercriticalsection

DICA: Isso é apenas um exemplo, em um binário que apresenta apenas um “Hello Word” na tela, existem muito mais importações de bibliotecas e funções que esses apresentados acima.

Agora vamos gerar o *hash* md5 dessa *string* acima.

```
hugo@ubuntu:~/Downloads$ echo -n  
"kernel32.deletcriticalsection,kernel32.entercriticalsection" | md5sum  
7b8469eda08899ccd072d0203d457a1d _  
hugo@ubuntu:~/Downloads$
```

Comando de prompt 2 – Gerando *imphash*
Fonte: Elaborado pelo autor (2020)

Arquivos com mesmo *imphash* não significa necessariamente que sejam do mesmo grupo de ameaças; talvez você precise correlacionar informações de várias fontes para classificar seu malware. Por exemplo, é possível que as amostras de malware tenham sido geradas usando um kit de compilador comum que é compartilhado entre grupos; nesses casos, as amostras podem ter o mesmo *imphash* (tradução do autor) (MONNAPPA; MONNAPPA, 2018).

2.3 VírusTotal

Virustotal é um website que executa um serviço gratuito de análise de arquivos bem como URLs e domínios, por meio do qual é possível detectar se arquivos possuem conteúdos maliciosos por scanners de antivírus, e se uma URL ou domínio são seguros ou não, utilizando *blacklists*. A grande vantagem de usar, inicialmente, o *Virustotal* para uma análise preliminar, é que ele utiliza cerca de 70 *scanners* de antivírus para analisar os arquivos enviados.

Para o envio de arquivos, o *Virustotal* oferece alguns métodos, por exemplo seu website em <www.virustotal.com>, no qual podemos fazer um upload de arquivo para ser analisado, aplicação para Windows, Linux e MAC; e podemos interagir diretamente com o *Virustotal* a partir do nosso sistema operacional, extensões para navegadores e uma API que suporta, dependendo da versão, diversas linguagens que vão de C ++ até GO, como clientes da API.

A interface da web tem a maior prioridade de digitalização entre os métodos de envio disponíveis publicamente. Os envios podem ser escritos em qualquer linguagem de programação usando a API pública baseada em HTTP. Assim como os arquivos, as URLs podem ser enviadas por vários métodos diferentes, incluindo a página da Web do VirusTotal, extensões do navegador e a API. Ao enviar um arquivo ou URL, os resultados básicos são compartilhados com o remetente e entre os parceiros examinadores, que usam os resultados para aprimorar seus próprios sistemas (tradução do autor) (TOTAL, 2020).

Como podemos observar, o serviço guarda informações dos arquivos que foram analisados para que sejam compartilhados entre outros usuários, ou seja, pelo ponto de vista do criador de malware, não é interessante testar seu malware no *Virustotal*, pois será gerada uma assinatura a ser compartilhada entre todos.

2.4 Ferramenta PEiD

Frequentemente, criadores de malwares ofuscam seus programas maliciosos utilizando *packers*, é aqui que entra uma ferramenta que consegue detectar não só *packers*, como também *cryptors* e compiladores em arquivos PE, possuindo mais de 600 assinaturas para isso.

Os mantenedores do PEiD descontinuaram o projeto, mas é possível fazer o download em: <https://softpedia-secure-download.com/dl/61d20bffddb5a5c2952c45ec40e0f09d/5f43d007/100004102/software/programming/PEiD-0.95-20081103.zip>.

2.5 Ferramenta pestudio

Ferramenta amplamente utilizada por profissionais de segurança, seu desenvolvimento iniciou em 2009, sendo mundialmente apoiado por analistas de malware. Além de sua versão paga, existe a versão gratuita que disponibiliza interessantes *features* para serem usadas.

O pestudio gratuito encontra-se disponível para download em: <https://www.winitor.com/download>.

3 ANÁLISE ESTÁTICA AVANÇADA

A partir de agora, evoluiremos um pouco mais na análise estática. Vamos descobrir como uma função é formada na memória RAM, como a pilha é estruturada, bem como seus parâmetros são inseridos na pilha (stack).

Análise estática avançada e engenharia reversa de malware são bastante abrangentes. Existem livros que se dedicam apenas à engenharia reversa, então, focaremos na chamada de função, que todo e qualquer programa executará e terá sempre o mesmo padrão em código de instruções de máquina.

3.1 Função

Quando executamos um software, seja ele malicioso ou não, o sistema operacional lerá todas as instruções de códigos salvos no disco e as carregará na memória principal, memória RAM. Quando carregado na RAM, esse software terá a estrutura mostrada na Figura “Esboço de um programa na RAM”.

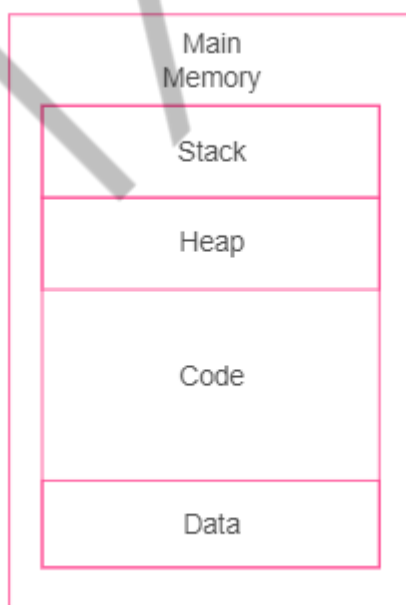


Figura 1 – Esboço de um programa na RAM
Fonte: Google Imagens (2019)

Dados: Usados para armazenamento de valores que não podem ser alterados durante a execução do programa; são conhecidos como valores globais, porque estarão disponíveis para qualquer parte do programa.

Código: Controla o que a CPU deverá executar e todas as suas tarefas.

Heap: Conhecida como memória dinâmica, pois é usada para criar e remover os valores de que o programa precisar, seu conteúdo é frequentemente alterado durante a execução do programa.

Pilha: É o local onde são armazenadas as funções, quando são chamadas pelo programa e utiliza a estrutura LIFO (do inglês *Last In, First Out*) ou seja, o último que entra na pilha será o primeiro a sair, exatamente como em uma pilha de pratos, não é possível remover qualquer prato da pilha se não começar pelo último inserido.

A partir desse momento, nos concentraremos na pilha (*stack*), como ela é formada e utilizada.

Os registradores que suportam a *stack* são o ESP e EBP.

- O ESP (*stack pointer*) é o ponteiro para o topo da pilha, ou seja, o endereço de memória que aponta para o topo da pilha e seu valor é alterado sempre que algum dado é inserido ou removido da pilha.
- O EBP (*base pointer*) é o ponteiro para a base da pilha, sempre que o programa se referir a algum dado dentro da pilha, usará este ponteiro para acessar esse dado, por exemplo, quando o processador se referir a uma variável local, ela estará representada por $[EBP - x]$, sendo x a posição da memória relativa ao dado na *stack*.

Funções são trechos de códigos que realizam tarefas específicas, estão separadas do código principal e são acionadas ao longo da execução do programa, podendo ser chamadas mais de uma vez. Por exemplo, no trecho de código “Código em C com chamada de função”, temos a função *main*, linha 07, função principal da linguagem C, todo e qualquer software desenvolvido em C iniciará com ela, dentro da nossa função principal, temos a chamada da função *soma*, linha 08 com dois parâmetros, sendo passado para a função e, na linha 03, temos o início da função *soma*.

```
01#include <stdio.h>
02
03int soma(int x, int y) {
04    return x+y;
05}
06
```

```
07int main(void) {  
08    printf("A soma de 3 + 4 e: %d\n", soma(3,4));  
09    return 0;  
10}
```

Código-fonte 1 – Código em C com chamada de função
Fonte: Elaborado pelo autor (2020)

Principais características da *stack* (pilha):

- Para cada função chamada dentro do código escrito, existirá uma *call* no código compilado e uma nova *stack* será criada.
- É usada para armazenar dados e variáveis das funções, que serão manipulados até a finalização da função.
- Argumentos (variáveis) são inseridos na pilha antes da chamada (*call*) da função.
- As instruções em assembly para a pilha são "PUSH (insere dados na pilha) e POP (remove dados da pilha)" para o compilador Microsoft Visual Studio. No caso do GCC, o PUSH é substituído pelo MOV.
- Antes de a função ser chamada, existe o prólogo, no qual são preparados a pilha e os registradores que serão utilizados pela variável local.
- Após término da execução da função existe o epílogo, no qual a pilha é restaurada, o ESP é liberado e o EBP é liberado para as próximas funções.
- A pilha é alocada no formato *top-down*, ou seja, ela aumentará sempre para os endereços de memória menores.

CONCLUSÃO

Neste capítulo, já conseguimos obter algumas informações importantes acerca de um artefato, por exemplo, data de criação, *imphash*, bibliotecas e funções utilizadas pelo malware na API do Windows entre outras. É de extrema importância que um analista de malware conheça a API do Windows, suas bibliotecas, funções e parâmetros, pois, em uma análise estática avançada (análise de código), dependendo da quantidade de parâmetros de cada função importada, o analista saberá, com exatidão, qual função está sendo chamada naquele determinado momento. Também

conhecemos algumas ferramentas, porém existem outras que o aluno pode e deve buscar para conhecer e assim se adaptar à sua melhor escolha.

Sobre a análise estática avançada, caso o aluno queira se embrenhar nessa área, o ideal é iniciar com os manuais da arquitetura Intel x86 disponíveis em <<https://software.intel.com/content/www/br/pt/develop/articles/intel-sdm.html>> e alguns livros na bibliografia desta disciplina, como o *Reversing: Secrets of Reverse Engineering* de Eldad Eilam.

Boa sorte com os seus estudos!

EMANIP

REFERÊNCIAS

MONNAPPA, A. K.; MONNAPPA, A. K. **Learning Malware Analysis**. Packt Publishing, 2018.

VIRUS TOTAL. 2020. Disponível em: <<https://support.virustotal.com/hc/en-us/articles/115002126889-How-it-works>>. Acesso em: 23 set. 2020.

EMANIP